

Lab Preparation

```
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$ python3 --version
Python 3.12.3
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$ openssl version
OpenSSL 3.0.13 30 Jan 2024 (Library: OpenSSL 3.0.13 30 Jan 2024)
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$ pip show cryptography pytest
Name: cryptography
Version: 49.0.0
Summary: cryptography is a package which provides cryptographic recipes and primitives to Python developers.
Home-page: https://github.com/pyca/cryptography
Author:
Author-email: The Python Cryptographic Authority and individual contributors <cryptography-dev@python.org>
License-Expression: Apache-2.0 OR BSD-3-Clause
Location: /home/ubuntu/csce465-agentsec/.venv/lib/python3.12/site-packages
Requires: cffi
Required-by:
---
Name: pytest
Version: 9.1.1
Summary: pytest: simple powerful testing with Python
Home-page: https://docs.pytest.org/en/latest/
Author: Brianna Laughner, Bruno Oliveira, Floris Bruynooghe, Freya Bruhin, Holger Krekel, Others (See AUTHORS), Ronny Pfannschmidt
Author-email:
License-Expression: MIT
Location: /home/ubuntu/csce465-agentsec/.venv/lib/python3.12/site-packages
Requires: iniconfig, packaging, pluggy, pygments
Required-by:
```

Task 1

```
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$ python3 baseline_ctr.py
=====
SENDER
=====
plaintext command      : {"action":"READ","path":"notes.txt"}
ciphertext (on wire)   : 45d4f989e4a7d66d4c3deea50d1e89c3a0b575e2b326ca18628c332b1f3fa5978115913e
=====
RELAY (attacker: has NO key, NO keystream)
=====
target offset         : 11
original bytes P      : b'READ'  52454144
desired bytes P'      : b'EDIT'  45444954

Per-byte XOR relation C'[i] = C[i] XOR (P[i] XOR P'[i]):
idx P  P' delta=P^P'  C[i]  C'[i]=C[i]^delta
11 52  45      17    a5    b2
12 45  44      01    0d    0c
13 41  49      08    1e    16
14 44  54      10    89    99

ciphertext in         : 45d4f989e4a7d66d4c3deea50d1e89c3a0b575e2b326ca18628c332b1f3fa5978115913e
ciphertext out        : 45d4f989e4a7d66d4c3deeb20c1699c3a0b575e2b326ca18628c332b1f3fa5978115913e
=====
RECEIVER (receives TAMPERED message)
=====
decrypted              : {"action":"EDIT","path":"notes.txt"}
>>> EXECUTING action 'EDIT' on 'notes.txt'

### REPLAY: attacker resends the original ciphertext unchanged ###

=====
RECEIVER (1st delivery of original)
=====
decrypted              : {"action":"READ","path":"notes.txt"}
>>> EXECUTING action 'READ' on 'notes.txt'

=====
RECEIVER (2nd delivery of original <-- processed AGAIN)
=====
decrypted              : {"action":"READ","path":"notes.txt"}
>>> EXECUTING action 'READ' on 'notes.txt'
```

I built a sender to relay to receiver pipeline around the AES-CTR encryption with no MAC. The sender and the receiver share a secret key and nonce while the relay simulates a third party that can read and rewrite bytes but doesn't have the key. The receiver is made stateless so that it decrypts and acts on any valid command. The command used is `{"action":"READ","path":"notes.txt"}` as required by the assignment.

In the experiment, the relay changed the command's action from "READ" to "EDIT" by using XOR on the ciphertext at a known offset, flipping only the relevant four bytes, without needing to recover the key. The receiver accepted and executed the tampered command. Resending the original ciphertext resulted in the receiver processing the same command twice as expected. Both intended actions succeeded without errors nor warnings from the receiver.

The root cause of this result is that the encryption provides only confidentiality, meaning that it hides what the message says but does not guarantee whether the message was altered or

fresh. AES-CTR's use of XOR makes it malleable in that a bit-flip in the ciphertext will cause the same bit-flip in the plaintext. Due to the ciphertext having no proof of origin or uniqueness, an identical copy is indistinguishable from a new message.

Task 2

In my design, the gateway and node each send a public value $g^x \bmod p$ and combine it with their own private exponent to establish the secret shared key Z which never crosses the channel and seeds K_master , demonstrating Diffie-Hellman key establishment. Each party signs role || SHA-256(transcript) and only the holder of a party's long-term RSA private key can produce a digital signature its pre-distributed public key will verify. Combined with an explicit check, completing the handshake verifies that you are speaking to the intended person, showing entity authentication. Fresh 16-byte nonces get folded into the transcript so every session's TH is unique. A handshake recorded off one session and replayed into another will produce a TH different from what an honest party computes from fresh values so the replayed signatures will fail, displaying replay resistance. Long-term RSA keys only sign, never encrypting or transporting the session keys which come entirely from the ephemeral DH secret shared key Z , and ephemeral private exponents are discarded after the session so even if the RSA private key is stolen later, past Z values cannot be reconstructed. Compromise of the identity key does not retroactively break recorded sessions, showing forward secrecy. Finally, instead of reusing one key everywhere, the KDF derives four independent keys with distinct labels for gateway-to-node, node-to-gateway, and encryption vs MAC keys, each as an HMAC of K_master . Because HMAC outputs under different labels are computationally independent, weakness or misuse of one key cannot contaminate another and the directional split removes any chance of one side's traffic being replayed as another's.

The reason why signing identities, roles, nonces, and Diffie-Hellman values stop man-in-the-middle and reflection attacks is first due to the fact that man-in-the-middle attacks work by substituting the attacker's public value for each party's but here, the Diffie-Hellman public values are inside a signed transcript so any substitution forces the attacker to produce a fresh signature over an altered transcript which requires an honest party's RSA private key or binds the attacker's own identity. Binding the identities which ties the authenticated name to the exact keys in use, binding the nonces which makes each session's signed material unique, and binding the role defeats reflection attacks because if the attacker bounces a gateway's own message back, it will carry the gateway role or identity but the receiver is verifying against the peer's role and identity so it gets rejected. If the signatures are removed, then only the raw Diffie-Hellman values plus nonces are left with no authentication. Confidentiality against a passive eavesdropper survives as Z remains hidden but there is no defense against an active attacker since a man-in-the-middle runs two separate Diffie-Hellman exchanges, one with each side using its own public values, then transparently relays and reads everything. Additionally, it becomes open to reflection attacks as well since nothing distinguishes the two roles.

Task 4

Attacker Model and Non-Goals

Assuming a Dolev-Yao network attacker that sits between the gateway and node with full control of the channel, being able to observe, drop, reorder, replay, modify, inject records, and initiate handshakes. The attacker does not hold either party's long-term RSA private key and cannot break underlying primitives such as AES-256, SHA-256, HMAC-SHA-256, or the discrete-log problem. Goals against this attacker are confidentiality and integrity of application records, mutual entity authentication, replay resistance, direction authenticity, and forward secrecy. Explicit non-goals for this are not hiding traffic metadata such as record lengths, timing, or message counts, not providing anonymity or availability guarantee, and not defending against endpoint compromises or side channels beyond the constant-time MAC comparison. Additionally, deciding whether an authenticated peer is permitted to perform a requested action is out of scope.

Why CTR requires unique IVs

Since AES-CTR produces a keystream by encrypting a counter sequence and applying XOR to it with the plaintext, the keystream is a pure function of (key, IV). If two records happen to be encrypted under the same key and IV, the same keystream gets reused and applying XOR to the two ciphertexts cancels it, revealing the XOR of the two plaintexts which is usually enough to recover both. By making the IV unique with "session_id || sequence", the sequence becomes strictly increasing and unique within a direction, each direction having its own key, thus making reuse impossible. Therefore, the sequence counter is not only used for anti-replay, but is also a hard requirement for confidentiality.

Why verify-before-decrypt matters

This is implemented where the record layer verifies the HMAC tag over "header || iv || ciphertext" before it runs AES-CTR. Since CTR is malleable, meaning flipping a ciphertext bit flips the matching plaintext bit, decrypting attacker-chosen bits first would enable chosen-ciphertext and oracle attacks where parse errors, timing, or downstream behavior would slowly leak the plaintext. By checking the tag first and never releasing the plaintext on failure, the decryptor only processes bytes that have been proven to be authentic. This guarantee is usually provided with Encrypt-then-MAC composed with verify-before-decrypt.

Why independent and MAC keys are used

By using one key for both AES and HMAC, it couples two primitives whose security analyses each assume an independently chosen key so their interaction becomes unproven and potentially dangerous. In the implementation, separate keys K_{enc} , K_{mac} , and one for each direction are derived from distinct KDF labels, making all four keys computationally independent from each other. Direction separation additionally makes it so that records authenticated with the gateway-to-node MAC key cannot be verified under the node-to-gateway key so reflection becomes blocked even before the direction byte is inspected.

Comparison with AES-GCM

The implementation includes Encrypt-then-MAC which is essentially the manual equivalent of a modern AEAD mode like AES-GCM. GCM pairs CTR-mode encryption with a built-in tag, GHASH, over the ciphertext as well as associated data which delivers confidentiality and integrity in a single primitive and is typically faster with less room for composition errors. Additionally, the unique-IV rule is stricter for GCM since nonce reuse will not only leak the plaintext but also the authentication subkey which enables forgeries.

Why a valid protected record may still contain a dangerous authorized tool call

Cryptography only authenticates the channel and not the intent behind any messages. A record can decrypt cleanly, pass its MAC, carry the exact expected sequence and direction, and originate from a correctly authenticated peer and still contain any authorized-but-harmful command such as {"action": "DELETE", "path": "keys.db"}. The protocol implemented only guarantees that the message sent is exactly what the authenticated peer sent, unmodified and unreplayed, but says nothing about whether the peer should be allowed to perform the action sent or whether the peer itself has been compromised. The job of defending against dangerous authorized calls belongs to the application above the record layer where authorization checks, input validation, least privilege, rate limiting, and auditing would be done.